

ig - SYNTAX ERROR

St x i v e n

a2z

DSA SHEET

- "495"

By - SYNTAX ERROR

Step 10: Sliding Window and Two Pointer

The PDF created by Abhishek Rathor (Instagram:SYNTAX ERROR)

PROBLEM:1

Problem Statement:

Write a Java program to find the **Longest Substring Without Repeating Characters**.

Java Code:

```
java
CopyEdit
import java.util.*;

public class LongestUniqueSubstring {
    public static void main(String[] args) {
        String s = "abcabcbb";
        System.out.println(lengthOfLongestSubstring(s));
    }

    public static int lengthOfLongestSubstring(String s) {
        Set<Character> set = new HashSet<>();
        int left = 0, right = 0, maxLen = 0;
        while (right < s.length()) {
            if (!set.contains(s.charAt(right))) {
                set.add(s.charAt(right));
                maxLen = Math.max(maxLen, right - left + 1);
                right++;
            } else {
                set.remove(s.charAt(left));
                left++;
            }
        }
        return maxLen;
    }
}
```

Explanation:

Use a sliding window with a set to track unique characters. Expand right pointer, shrink left when duplicates are found.

PROBLEM:2

Problem Statement:

Write a Java program for **Max Consecutive Ones III**. You are allowed to flip at most k 0s.

Java Code:

```
java
CopyEdit
public class MaxConsecutiveOnesIII {
    public static void main(String[] args) {
        int[] nums = {1,1,1,0,0,0,1,1,1,1,0};
        int k = 2;
        System.out.println(longestOnes(nums, k));
    }

    public static int longestOnes(int[] nums, int k) {
        int left = 0, right = 0, zeroCount = 0, maxLen = 0;
        while (right < nums.length) {
            if (nums[right] == 0) zeroCount++;
            while (zeroCount > k) {
                if (nums[left] == 0) zeroCount--;
                left++;
            }
            maxLen = Math.max(maxLen, right - left + 1);
            right++;
        }
        return maxLen;
    }
}
```

Explanation:

Sliding window with a counter for 0s. Shrink the window when the number of 0s exceeds k.

PROBLEM:3

Problem Statement:

Write a Java program for **Fruit Into Baskets**. You can only carry two types of fruits.

Java Code:

```
java
CopyEdit
import java.util.*;

public class FruitsIntoBaskets {
    public static void main(String[] args) {
        int[] fruits = {1,2,1,2,3};
        System.out.println(totalFruit(fruits));
    }

    public static int totalFruit(int[] fruits) {
        Map<Integer, Integer> map = new HashMap<>();
        int left = 0, maxLen = 0;

        for (int right = 0; right < fruits.length; right++) {
            map.put(fruits[right], map.getOrDefault(fruits[right], 0) + 1);
```

```

        while (map.size() > 2) {
            map.put(fruits[left], map.get(fruits[left]) - 1);
            if (map.get(fruits[left]) == 0) map.remove(fruits[left]);
            left++;
        }
        maxLen = Math.max(maxLen, right - left + 1);
    }

    return maxLen;
}
}

```

Explanation:

Use hashmap to count fruit types. Maintain window with max 2 types.

PROBLEM:4

Problem Statement:

Write a Java program for **Longest Repeating Character Replacement** with at most k replacements.

Java Code:

```

java
CopyEdit
public class LongestRepeatingCharReplacement {
    public static void main(String[] args) {
        String s = "AABABBA";
        int k = 1;
        System.out.println(characterReplacement(s, k));
    }

    public static int characterReplacement(String s, int k) {
        int[] freq = new int[26];
        int left = 0, maxFreq = 0, maxLen = 0;

        for (int right = 0; right < s.length(); right++) {
            freq[s.charAt(right) - 'A']++;
            maxFreq = Math.max(maxFreq, freq[s.charAt(right) - 'A']);

            while (right - left + 1 - maxFreq > k) {
                freq[s.charAt(left) - 'A']--;
                left++;
            }

            maxLen = Math.max(maxLen, right - left + 1);
        }

        return maxLen;
    }
}

```

Explanation:

Window maintains frequency of most common character. If replacements exceed k , shrink the window.

PROBLEM:5**Problem Statement:**

Write a Java program for **Binary Subarray With Sum**. Count subarrays with $\text{sum} = \text{goal}$.

Java Code:

```
java
CopyEdit
public class BinarySubarrayWithSum {
    public static void main(String[] args) {
        int[] nums = {1,0,1,0,1};
        int goal = 2;
        System.out.println(numSubarraysWithSum(nums, goal));
    }

    public static int numSubarraysWithSum(int[] nums, int goal) {
        return atMost(nums, goal) - atMost(nums, goal - 1);
    }

    private static int atMost(int[] nums, int S) {
        if (S < 0) return 0;
        int left = 0, res = 0, sum = 0;

        for (int right = 0; right < nums.length; right++) {
            sum += nums[right];
            while (sum > S) sum -= nums[left++];
            res += right - left + 1;
        }

        return res;
    }
}
```

Explanation:

Use prefix sum logic. Count subarrays with $\text{sum} \leq \text{goal}$ and subtract with $\text{sum} \leq \text{goal}-1$.

PROBLEM:6**Problem Statement:**

Write a Java program to **Count Number of Nice Subarrays** having exactly k odd numbers.

Java Code:

```
java
CopyEdit
public class NiceSubarrays {
    public static void main(String[] args) {
        int[] nums = {1,1,2,1,1};
        int k = 3;
        System.out.println(numberOfSubarrays(nums, k));
    }

    public static int numberOfSubarrays(int[] nums, int k) {
        return atMost(nums, k) - atMost(nums, k - 1);
    }

    private static int atMost(int[] nums, int k) {
        int left = 0, count = 0, res = 0;
        for (int right = 0; right < nums.length; right++) {
            if (nums[right] % 2 == 1) k--;
            while (k < 0) {
                if (nums[left++] % 2 == 1) k++;
            }
            res += right - left + 1;
        }
        return res;
    }
}
```

Explanation:

Sliding window to count subarrays with at most k and $k-1$ odd numbers.

PROBLEM:7

Problem Statement:

Write a Java program to count **Number of Substrings Containing All Three Characters 'a', 'b', and 'c'**.

Java Code:

```
java
CopyEdit
public class SubstringsWithAllChars {
    public static void main(String[] args) {
        String s = "abcabc";
        System.out.println(numberOfSubstrings(s));
    }

    public static int numberOfSubstrings(String s) {
        int[] count = new int[3];
        int left = 0, res = 0;

        for (int right = 0; right < s.length(); right++) {
```

```

        count[s.charAt(right) - 'a']++;
        while (count[0] > 0 && count[1] > 0 && count[2] > 0) {
            res += s.length() - right;
            count[s.charAt(left++) - 'a']--;
        }
    }

    return res;
}
}

```

Explanation:

As soon as all characters 'a', 'b', 'c' exist in window, add number of substrings from right to end.

PROBLEM:8

Problem Statement:

Write a Java program for **Maximum Points You Can Obtain from Cards**. You can pick k cards from either end.

Java Code:

```

java
CopyEdit
public class MaxPointsFromCards {
    public static void main(String[] args) {
        int[] cardPoints = {1,2,3,4,5,6,1};
        int k = 3;
        System.out.println(maxScore(cardPoints, k));
    }

    public static int maxScore(int[] cardPoints, int k) {
        int n = cardPoints.length;
        int total = 0;
        for (int i = 0; i < k; i++) total += cardPoints[i];

        int max = total;
        for (int i = 0; i < k; i++) {
            total -= cardPoints[k - 1 - i];
            total += cardPoints[n - 1 - i];
            max = Math.max(max, total);
        }

        return max;
    }
}

```

Explanation:

Try all combinations of left and right k picks and return maximum score.

PROBLEM:9

Problem Statement:

Write a Java program to find the **Longest Substring with At Most K Distinct Characters**.

Java Code:

```
java
CopyEdit
import java.util.*;

public class LongestKDistinctSubstring {
    public static void main(String[] args) {
        String s = "eceba";
        int k = 2;
        System.out.println(lengthOfLongestSubstringKDistinct(s, k));
    }

    public static int lengthOfLongestSubstringKDistinct(String s, int k) {
        if (s.length() == 0 || k == 0) return 0;

        Map<Character, Integer> map = new HashMap<>();
        int left = 0, maxLen = 0;

        for (int right = 0; right < s.length(); right++) {
            map.put(s.charAt(right), map.getOrDefault(s.charAt(right), 0) + 1);

            while (map.size() > k) {
                map.put(s.charAt(left), map.get(s.charAt(left)) - 1);
                if (map.get(s.charAt(left)) == 0)
                    map.remove(s.charAt(left));
                left++;
            }

            maxLen = Math.max(maxLen, right - left + 1);
        }

        return maxLen;
    }
}
```

Explanation:

Use a sliding window with a hashmap to track character frequencies. Shrink window when unique characters exceed k.

PROBLEM:10

Problem Statement:

Write a Java program to find the **Subarrays With Exactly K Different Integers**.

Java Code:

```
java
CopyEdit
import java.util.*;

public class SubarraysWithKDistinct {
    public static void main(String[] args) {
        int[] nums = {1,2,1,2,3};
        int k = 2;
        System.out.println(subarraysWithKDistinct(nums, k));
    }

    public static int subarraysWithKDistinct(int[] nums, int k) {
        return atMost(nums, k) - atMost(nums, k - 1);
    }

    private static int atMost(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        int left = 0, res = 0;

        for (int right = 0; right < nums.length; right++) {
            map.put(nums[right], map.getOrDefault(nums[right], 0) + 1);
            while (map.size() > k) {
                map.put(nums[left], map.get(nums[left]) - 1);
                if (map.get(nums[left]) == 0) map.remove(nums[left]);
                left++;
            }
            res += right - left + 1;
        }

        return res;
    }
}
```

Explanation:

Count subarrays with at most k distinct elements and subtract those with $k-1$ to get exactly k .

PROBLEM:11**Problem Statement:**

Write a Java program to find the **Minimum Window Substring** of s that contains all characters of t .

Java Code:

```

java
CopyEdit
import java.util.*;

public class MinimumWindowSubstring {
    public static void main(String[] args) {
        String s = "ADOBECODEBANC";
        String t = "ABC";
        System.out.println(minWindow(s, t));
    }

    public static String minWindow(String s, String t) {
        if (s.length() < t.length()) return "";

        Map<Character, Integer> map = new HashMap<>();
        for (char c : t.toCharArray())
            map.put(c, map.getOrDefault(c, 0) + 1);

        int left = 0, right = 0, count = map.size();
        int minLen = Integer.MAX_VALUE, start = 0;

        while (right < s.length()) {
            char c = s.charAt(right);
            if (map.containsKey(c)) {
                map.put(c, map.get(c) - 1);
                if (map.get(c) == 0) count--;
            }
            right++;

            while (count == 0) {
                if (right - left < minLen) {
                    minLen = right - left;
                    start = left;
                }
                char temp = s.charAt(left);
                if (map.containsKey(temp)) {
                    map.put(temp, map.get(temp) + 1);
                    if (map.get(temp) > 0) count++;
                }
                left++;
            }
        }

        return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start
+ minLen);
    }
}

```

Explanation:

Use sliding window and hash map. Expand to include all required chars, then shrink to find smallest valid window.

PROBLEM:12

Problem Statement:

Write a Java program for **Minimum Window Subsequence**. Find the minimum window in *s* which contains *t* as a subsequence.

Java Code:

```
java
CopyEdit
public class MinimumWindowSubsequence {
    public static void main(String[] args) {
        String s1 = "abcdebddde", s2 = "bde";
        System.out.println(minWindow(s1, s2));
    }

    public static String minWindow(String s1, String s2) {
        int m = s1.length(), n = s2.length();
        int start = -1, minLen = Integer.MAX_VALUE;

        for (int i = 0; i < m; i++) {
            if (s1.charAt(i) != s2.charAt(0)) continue;

            int j = 0, k = i;
            while (k < m && j < n) {
                if (s1.charAt(k) == s2.charAt(j)) j++;
                k++;
            }

            if (j == n) {
                int end = k - 1;
                j = n - 1;
                k = end;
                while (j >= 0) {
                    if (s1.charAt(k) == s2.charAt(j)) j--;
                    k--;
                }
                k++;
                if (end - k + 1 < minLen) {
                    minLen = end - k + 1;
                    start = k;
                }
            }
        }

        return start == -1 ? "" : s1.substring(start, start + minLen);
    }
}
```

Explanation:

Two-pointer approach: scan forward to find match, then scan backward to tighten window containing *t* as a subsequence.

Step 11: Heaps

PROBLEM:13

Problem Statement:

Write a Java program to demonstrate **Introduction to Priority Queues using Binary Heaps**. Implement a **Min-Heap** using `PriorityQueue`.

Java Code:

```
java
CopyEdit
import java.util.PriorityQueue;

public class MinHeapDemo {
    public static void main(String[] args) {
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        minHeap.add(10);
        minHeap.add(4);
        minHeap.add(15);
        minHeap.add(20);
        minHeap.add(0);

        while (!minHeap.isEmpty()) {
            System.out.print(minHeap.poll() + " ");
        }
    }
}
```

Explanation:

1. Java's `PriorityQueue` by default implements a min-heap.
 2. Elements are ordered based on their natural ordering.
 3. `poll()` retrieves and removes the smallest element.
-

PROBLEM:14

Problem Statement:

Write a Java program for **Min Heap and Max Heap Implementation** using `PriorityQueue`.

Java Code:

```
java
CopyEdit
import java.util.*;

public class HeapImplementation {
    public static void main(String[] args) {
```

```

// Min Heap
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
// Max Heap
PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Collections.reverseOrder());

int[] arr = {10, 3, 15, 7, 8};

for (int num : arr) {
    minHeap.add(num);
    maxHeap.add(num);
}

System.out.println("Min Heap:");
while (!minHeap.isEmpty()) {
    System.out.print(minHeap.poll() + " ");
}

System.out.println("\nMax Heap:");
while (!maxHeap.isEmpty()) {
    System.out.print(maxHeap.poll() + " ");
}
}
}

```

Explanation:

1. Min-heap uses default ordering.
 2. Max-heap is created using a reverse comparator via `Collections.reverseOrder()`.
-

PROBLEM:15

Problem Statement:

Write a Java program to **Check if an Array Represents a Min Heap or Not.**

Java Code:

```

java
CopyEdit
public class CheckMinHeap {
    public static void main(String[] args) {
        int[] heap = {1, 3, 5, 7, 9, 6};
        System.out.println(isMinHeap(heap, heap.length));
    }

    public static boolean isMinHeap(int[] arr, int n) {
        for (int i = 0; i <= (n - 2) / 2; i++) {
            if (arr[i] > arr[2 * i + 1]) return false;
            if (2 * i + 2 < n && arr[i] > arr[2 * i + 2]) return false;
        }
        return true;
    }
}

```

```
}
```

Explanation:

For each non-leaf node at index i , check if it's less than or equal to its children at $2i+1$ and $2i+2$.

PROBLEM:16**Problem Statement:**

Write a Java program to **Convert Min Heap to Max Heap**.

Java Code:

```
java
CopyEdit
import java.util.*;

public class ConvertMinToMaxHeap {
    public static void main(String[] args) {
        int[] minHeap = {1, 3, 5, 7, 9, 6};
        convertToMaxHeap(minHeap);
        System.out.println(Arrays.toString(minHeap));
    }

    public static void convertToMaxHeap(int[] arr) {
        int n = arr.length;
        for (int i = (n - 2) / 2; i >= 0; i--) {
            heapify(arr, n, i);
        }
    }

    public static void heapify(int[] arr, int n, int i) {
        int largest = i;
        int left = 2*i + 1;
        int right = 2*i + 2;

        if (left < n && arr[left] > arr[largest]) largest = left;
        if (right < n && arr[right] > arr[largest]) largest = right;

        if (largest != i) {
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;
            heapify(arr, n, largest);
        }
    }
}
```

Explanation:

1. Start from the last non-leaf node and heapify in reverse.

2. Convert structure to satisfy max-heap property using recursion.

PROBLEM:17

Problem Statement:

Write a Java program to find the **Next Greater Element** for each element in an array.

Java Code:

```
java
CopyEdit
import java.util.*;

public class NextGreaterElement {
    public static void main(String[] args) {
        int[] arr = {4, 5, 2, 10, 8};
        int[] result = new int[arr.length];
        Stack<Integer> stack = new Stack<>();

        for (int i = arr.length - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() <= arr[i]) {
                stack.pop();
            }
            result[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(arr[i]);
        }

        System.out.println("Next Greater Elements:");
        System.out.println(Arrays.toString(result));
    }
}
```

Explanation:

1. Iterate from right to left.
 2. Use a stack to keep track of potential next greater elements.
 3. Pop smaller elements and store the next greater (top of stack).
-

PROBLEM:18

Problem Statement:

Write a Java program to find the **Next Greater Element in a Circular Array** (Next Greater Element II).

Java Code:

```

java
CopyEdit
import java.util.*;

public class NextGreaterElementCircular {
    public static void main(String[] args) {
        int[] arr = {1, 2, 1};
        int n = arr.length;
        int[] result = new int[n];
        Stack<Integer> stack = new Stack<>();

        for (int i = 2 * n - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() <= arr[i % n]) {
                stack.pop();
            }
            result[i % n] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(arr[i % n]);
        }

        System.out.println("Next Greater Elements (Circular):");
        System.out.println(Arrays.toString(result));
    }
}

```

Explanation:

1. Traverse the array twice to simulate circularity.
 2. Use modulo to wrap indices.
-

PROBLEM:19

Problem Statement:

Write a Java program to find the **Next Smaller Element** for each element.

Java Code:

```

java
CopyEdit
import java.util.*;

public class NextSmallerElement {
    public static void main(String[] args) {
        int[] arr = {4, 8, 5, 2, 25};
        int[] result = new int[arr.length];
        Stack<Integer> stack = new Stack<>();

        for (int i = arr.length - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() >= arr[i]) {
                stack.pop();
            }
            result[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(arr[i]);
        }
    }
}

```



```

    }

    System.out.println("Next Smaller Elements:");
    System.out.println(Arrays.toString(result));
}
}

```

Explanation:

1. Traverse from right to left.
 2. Use a stack to find the next smaller element efficiently.
-

PROBLEM:20

Problem Statement:

Write a Java program to find the **Number of Next Greater Elements to the right** for each element in an array.

Java Code:

```

java
CopyEdit
import java.util.*;

public class CountNextGreater {
    public static void main(String[] args) {
        int[] arr = {4, 5, 2, 25};
        int[] count = new int[arr.length];

        for (int i = 0; i < arr.length; i++) {
            int c = 0;
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] > arr[i]) {
                    c++;
                }
            }
            count[i] = c;
        }

        System.out.println("Count of NGEs to the right:");
        System.out.println(Arrays.toString(count));
    }
}

```

Explanation:

1. Brute force method to count NGEs to the right.
2. Can be optimized with segment trees/fenwick trees.

PROBLEM:21

Problem Statement:

Write a Java program to compute the amount of **Trapped Rainwater** between bars of different heights.

Java Code:

```
java
CopyEdit
public class TrappingRainwater {
    public static void main(String[] args) {
        int[] height = {4, 2, 0, 3, 2, 5};
        System.out.println("Trapped Rainwater: " + trap(height));
    }

    public static int trap(int[] height) {
        int n = height.length;
        int[] leftMax = new int[n];
        int[] rightMax = new int[n];
        int water = 0;

        leftMax[0] = height[0];
        for (int i = 1; i < n; i++)
            leftMax[i] = Math.max(leftMax[i - 1], height[i]);

        rightMax[n - 1] = height[n - 1];
        for (int i = n - 2; i >= 0; i--)
            rightMax[i] = Math.max(rightMax[i + 1], height[i]);

        for (int i = 0; i < n; i++)
            water += Math.min(leftMax[i], rightMax[i]) - height[i];

        return water;
    }
}
```

Explanation:

1. Precompute left and right maximums for each bar.
 2. For each bar, calculate how much water it can trap.
 3. Sum up the water trapped at each index.
-

PROBLEM:22

Problem Statement:

Write a Java program to find the **sum of minimums of all subarrays**.

Java Code:

```
java
CopyEdit
import java.util.*;

public class SumOfSubarrayMinimums {
    public static void main(String[] args) {
        int[] arr = {3, 1, 2, 4};
        System.out.println("Sum of Subarray Minimums: " +
sumSubarrayMins(arr));
    }

    public static int sumSubarrayMins(int[] arr) {
        int mod = 1_000_000_007;
        int n = arr.length;
        Stack<Integer> stack = new Stack<>();
        int[] prevLess = new int[n];
        int[] nextLess = new int[n];

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && arr[stack.peek()] > arr[i])
stack.pop();
            prevLess[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(i);
        }

        stack.clear();
        for (int i = n - 1; i >= 0; i--) {
            while (!stack.isEmpty() && arr[stack.peek()] >= arr[i])
stack.pop();
            nextLess[i] = stack.isEmpty() ? n : stack.peek();
            stack.push(i);
        }

        long sum = 0;
        for (int i = 0; i < n; i++) {
            sum = (sum + (long)(i - prevLess[i]) * (nextLess[i] - i) *
arr[i]) % mod;
        }

        return (int) sum;
    }
}
```

Explanation:

1. For each element, calculate how many subarrays it's the minimum in.
 2. Use monotonic stack to find previous and next less elements.
 3. Multiply frequency with value and sum up.
-

PROBLEM:23

Problem Statement:

Write a Java program to simulate **Asteroid Collision**.

Java Code:

```
java
CopyEdit
import java.util.*;

public class AsteroidCollision {
    public static void main(String[] args) {
        int[] asteroids = {5, 10, -5};
        System.out.println("After Collision: " +
            Arrays.toString(asteroidCollision(asteroids)));
    }

    public static int[] asteroidCollision(int[] asteroids) {
        Stack<Integer> stack = new Stack<>();

        for (int a : asteroids) {
            while (!stack.isEmpty() && a < 0 && stack.peek() > 0) {
                if (stack.peek() < -a) stack.pop();
                else if (stack.peek() == -a) {
                    stack.pop();
                    a = 0;
                } else {
                    a = 0;
                }
            }
            if (a != 0) stack.push(a);
        }

        int[] result = new int[stack.size()];
        for (int i = stack.size() - 1; i >= 0; i--) result[i] = stack.pop();
        return result;
    }
}
```

Explanation:

1. Use stack to simulate collisions.
 2. Only opposite signs (+ve & -ve) interact.
 3. Compare sizes and decide which one survives.
-

PROBLEM:24**Problem Statement:**

Write a Java program to find the **sum of ranges of all subarrays**.

Java Code:

```
java
CopyEdit
public class SubarrayRanges {
    public static void main(String[] args) {
        int[] nums = {1, 2, 3};
        System.out.println("Sum of Subarray Ranges: " +
subArrayRanges(nums));
    }

    public static long subArrayRanges(int[] nums) {
        long sum = 0;
        for (int i = 0; i < nums.length; i++) {
            int max = nums[i], min = nums[i];
            for (int j = i; j < nums.length; j++) {
                max = Math.max(max, nums[j]);
                min = Math.min(min, nums[j]);
                sum += max - min;
            }
        }
        return sum;
    }
}
```

Explanation:

1. Brute force: Compute max and min in every subarray.
2. Add the range (max - min) to result.
3. Can be optimized using monotonic stacks.

PROBLEM:25

Problem Statement:

Write a Java program to **Remove K Digits** to make the smallest number.

Java Code:

```
java
CopyEdit
import java.util.*;

public class RemoveKDigits {
    public static void main(String[] args) {
        String num = "1432219";
        int k = 3;
        System.out.println("Smallest Number: " + removeKdigits(num, k));
    }

    public static String removeKdigits(String num, int k) {
        Stack<Character> stack = new Stack<>();

        for (char digit : num.toCharArray()) {
```

```

        while (!stack.isEmpty() && k > 0 && stack.peek() > digit) {
            stack.pop();
            k--;
        }
        stack.push(digit);
    }

    while (k-- > 0) stack.pop();

    StringBuilder sb = new StringBuilder();
    while (!stack.isEmpty()) sb.append(stack.pop());
    sb.reverse();

    while (sb.length() > 1 && sb.charAt(0) == '0') sb.deleteCharAt(0);

    return sb.length() == 0 ? "0" : sb.toString();
}
}

```

Explanation:

1. Use stack to build smallest possible number.
2. Pop digits if a smaller one is found and $k > 0$.
3. Remove leading zeros from result.

PROBLEM:26

Problem Statement:

Write a Java program to find the **Largest Rectangle in a Histogram**.

Java Code:

```

java
CopyEdit
import java.util.*;

public class LargestRectangleHistogram {
    public static void main(String[] args) {
        int[] heights = {2, 1, 5, 6, 2, 3};
        System.out.println("Largest Rectangle Area: " +
largestRectangleArea(heights));
    }

    public static int largestRectangleArea(int[] heights) {
        Stack<Integer> stack = new Stack<>();
        int maxArea = 0;
        int n = heights.length;
        for (int i = 0; i <= n; i++) {
            int currHeight = (i == n) ? 0 : heights[i];
            while (!stack.isEmpty() && currHeight < heights[stack.peek()])
{
                int height = heights[stack.pop()];
                int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                maxArea = Math.max(maxArea, height * width);
            }
        }
    }
}

```

```

        }
        stack.push(i);
    }
    return maxArea;
}
}

```

Explanation:

1. Use a monotonic increasing stack.
 2. For every bar, calculate area with it as the smallest height.
 3. Maintain maximum area encountered.
-

PROBLEM:27

Problem Statement:

Write a Java program to find the **Maximal Rectangle** in a 2D binary matrix.

Java Code:

```

java
CopyEdit
public class MaximalRectangle {
    public static void main(String[] args) {
        char[][] matrix = {
            {'1','0','1','0','0'},
            {'1','0','1','1','1'},
            {'1','1','1','1','1'},
            {'1','0','0','1','0'}
        };
        System.out.println("Maximal Rectangle Area: " +
            maximalRectangle(matrix));
    }

    public static int maximalRectangle(char[][] matrix) {
        if (matrix.length == 0) return 0;
        int n = matrix[0].length;
        int[] heights = new int[n];
        int maxArea = 0;

        for (char[] row : matrix) {
            for (int i = 0; i < n; i++) {
                heights[i] = (row[i] == '1') ? heights[i] + 1 : 0;
            }
            maxArea = Math.max(maxArea, largestRectangleArea(heights));
        }

        return maxArea;
    }

    public static int largestRectangleArea(int[] heights) {

```

```

Stack<Integer> stack = new Stack<>();
int max = 0;
int n = heights.length;
for (int i = 0; i <= n; i++) {
    int h = (i == n) ? 0 : heights[i];
    while (!stack.isEmpty() && h < heights[stack.peek()]) {
        int height = heights[stack.pop()];
        int width = stack.isEmpty() ? i : i - stack.peek() - 1;
        max = Math.max(max, height * width);
    }
    stack.push(i);
}
return max;
}
}

```

Explanation:

1. Treat each row as base of histogram.
2. Use histogram rectangle approach for each row.
3. Maintain and return max area found.

PROBLEM:28

Problem Statement:

Design a simplified version of Twitter where users can post tweets, follow/unfollow other users, and see the 10 most recent tweets in their news feed.

Java Code:

```

java
CopyEdit
import java.util.*;

class Twitter {

    private static class Tweet {
        int time;
        int tweetId;

        Tweet(int time, int tweetId) {
            this.time = time;
            this.tweetId = tweetId;
        }
    }

    private Map<Integer, Set<Integer>> following;
    private Map<Integer, List<Tweet>> tweets;
    private int time;

    public Twitter() {
        following = new HashMap<>();
        tweets = new HashMap<>();
        time = 0;
    }
}

```



```

    }

    public void postTweet(int userId, int tweetId) {
        tweets.putIfAbsent(userId, new ArrayList<>());
        tweets.get(userId).add(new Tweet(time++, tweetId));
    }

    public List<Integer> getNewsFeed(int userId) {
        PriorityQueue<Tweet> pq = new PriorityQueue<>((a, b) -> b.time -
a.time);
        Set<Integer> users = new HashSet<>();
        users.add(userId);
        users.addAll(following.getOrDefault(userId, new HashSet<>()));

        for (int uid : users) {
            List<Tweet> userTweets = tweets.getOrDefault(uid, new
ArrayList<>());
            for (int i = Math.max(0, userTweets.size() - 10); i <
userTweets.size(); i++) {
                pq.add(userTweets.get(i));
            }
        }

        List<Integer> res = new ArrayList<>();
        int count = 0;
        while (!pq.isEmpty() && count++ < 10) {
            res.add(pq.poll().tweetId);
        }
        return res;
    }

    public void follow(int followerId, int followeeId) {
        following.putIfAbsent(followerId, new HashSet<>());
        following.get(followerId).add(followeeId);
    }

    public void unfollow(int followerId, int followeeId) {
        if (following.containsKey(followerId)) {
            following.get(followerId).remove(followeeId);
        }
    }
}

```

Explanation:

1. Tweets stored per user with timestamps.
 2. Use a max heap to fetch the top 10 recent tweets.
 3. Users can follow/unfollow others.
 4. News feed combines tweets of user + followed users.
-

PROBLEM:29

Problem Statement:

Given n ropes with different lengths, connect them with minimum cost. The cost to connect two ropes is the sum of their lengths.

Java Code:

```
java
CopyEdit
import java.util.*;

public class ConnectRopes {
    public static void main(String[] args) {
        int[] ropes = {4, 3, 2, 6};
        System.out.println("Minimum Cost: " + minCost(ropes));
    }

    public static int minCost(int[] ropes) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        for (int rope : ropes) {
            pq.add(rope);
        }

        int cost = 0;
        while (pq.size() > 1) {
            int first = pq.poll();
            int second = pq.poll();
            cost += first + second;
            pq.add(first + second);
        }

        return cost;
    }
}
```

Explanation:

1. Use a min heap (priority queue) to always combine the shortest two ropes.
 2. Keep track of total cost.
 3. Repeat until one rope remains.
-

PROBLEM:30**Problem Statement:**

Find the k th largest element in a stream of integers.

Java Code:

```

java
CopyEdit
import java.util.*;

class KthLargest {
    private PriorityQueue<Integer> minHeap;
    private int k;

    public KthLargest(int k, int[] nums) {
        this.k = k;
        minHeap = new PriorityQueue<>();
        for (int num : nums) {
            add(num);
        }
    }

    public int add(int val) {
        minHeap.add(val);
        if (minHeap.size() > k)
            minHeap.poll();
        return minHeap.peek();
    }
}

```

Explanation:

1. Maintain a min heap of size k .
2. Always keep k largest elements.
3. Top of heap is the k th largest element.

PROBLEM: 26

Problem Statement:

Write a Java program to convert an Infix expression to Postfix using Stack.

Java Code:

```

java
CopyEdit
import java.util.*;

public class InfixToPostfix {
    public static int precedence(char ch) {
        switch (ch) {
            case '+': case '-': return 1;
            case '*': case '/': return 2;
            case '^': return 3;
        }
        return -1;
    }
}

```

```

public static String convert(String exp) {
    StringBuilder result = new StringBuilder();
    Stack<Character> stack = new Stack<>();

    for (int i = 0; i < exp.length(); i++) {
        char c = exp.charAt(i);

        if (Character.isLetterOrDigit(c)) {
            result.append(c);
        } else if (c == '(') {
            stack.push(c);
        } else if (c == ')') {
            while (!stack.isEmpty() && stack.peek() != '(')
                result.append(stack.pop());
            stack.pop();
        } else {
            while (!stack.isEmpty() && precedence(c) <=
precedence(stack.peek()))
                result.append(stack.pop());
            stack.push(c);
        }
    }

    while (!stack.isEmpty())
        result.append(stack.pop());

    return result.toString();
}

public static void main(String[] args) {
    String infix = "a+b*(c^d-e)^(f+g*h)-i";
    System.out.println("Postfix: " + convert(infix));
}
}

```

Explanation:

1. Traverse the infix expression.
 2. Add operands to the result.
 3. Use stack to maintain operator precedence and handle parentheses.
 4. Pop any remaining operators to result.
-

PROBLEM:31

Problem Statement:

Given two integer arrays A and B of size N, find the maximum sum combination from all the possible combinations of sum of one element from A and one from B. Return the top K maximum sum combinations.

Java Code:

```

java
CopyEdit
import java.util.*;

public class MaxSumCombinations {
    public static List<Integer> maxCombinations(int[] A, int[] B, int K) {
        Arrays.sort(A);
        Arrays.sort(B);
        int N = A.length;

        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> b[0] -
a[0]);
        Set<String> set = new HashSet<>();

        pq.add(new int[]{A[N - 1] + B[N - 1], N - 1, N - 1});
        set.add((N - 1) + "_" + (N - 1));

        List<Integer> result = new ArrayList<>();
        while (K-- > 0 && !pq.isEmpty()) {
            int[] top = pq.poll();
            result.add(top[0]);

            int i = top[1], j = top[2];
            if (i - 1 >= 0 && set.add((i - 1) + "_" + j)) {
                pq.add(new int[]{A[i - 1] + B[j], i - 1, j});
            }

            if (j - 1 >= 0 && set.add(i + "_" + (j - 1))) {
                pq.add(new int[]{A[i] + B[j - 1], i, j - 1});
            }
        }

        return result;
    }

    public static void main(String[] args) {
        int[] A = {1, 4, 2, 3};
        int[] B = {2, 5, 1, 6};
        int K = 4;
        System.out.println(maxCombinations(A, B, K));
    }
}

```

Explanation:

1. Sort both arrays.
 2. Use a max heap to get largest pair sums.
 3. Avoid duplicates using a set.
 4. Extract top K combinations.
-

PROBLEM:32

Problem Statement:

Find the median from a stream of integers. The median is the middle value in an ordered integer list.

Java Code:

```
java
CopyEdit
import java.util.*;

class MedianFinder {
    private PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Collections.reverseOrder());
    private PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    public void addNum(int num) {
        maxHeap.offer(num);
        minHeap.offer(maxHeap.poll());

        if (maxHeap.size() < minHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }

    public double findMedian() {
        if (maxHeap.size() == minHeap.size()) {
            return (maxHeap.peek() + minHeap.peek()) / 2.0;
        } else {
            return maxHeap.peek();
        }
    }
}
```

Explanation:

1. Use two heaps: maxHeap for left half, minHeap for right.
 2. Balance their sizes while adding numbers.
 3. If even size, median is average of tops.
 4. If odd, median is top of maxHeap.
-

PROBLEM:33**Problem Statement:**

Return the k most frequent elements from the given integer array.

Java Code:

```
java
CopyEdit
import java.util.*;

public class KMostFrequent {
```

```

    public static List<Integer> topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int num : nums)
            map.put(num, map.getOrDefault(num, 0) + 1);

        PriorityQueue<Map.Entry<Integer, Integer>> pq = new
        PriorityQueue<>(
            Comparator.comparingInt (Map.Entry::getValue)
        );

        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            pq.add(entry);
            if (pq.size() > k)
                pq.poll();
        }

        List<Integer> result = new ArrayList<>();
        while (!pq.isEmpty())
            result.add(pq.poll().getKey());

        Collections.reverse(result);
        return result;
    }

    public static void main(String[] args) {
        int[] nums = {1,1,1,2,2,3};
        int k = 2;
        System.out.println(topKFrequent(nums, k)); // Output: [1, 2]
    }
}

```

Explanation:

1. Count frequencies using a map.
2. Use a min heap to keep top k frequent.
3. Return the k most frequent keys.

Step 12: Greedy Algorithms

PROBLEM:34

Problem Statement:

You are given two integer arrays g (children's greed) and s (size of cookies). Assign cookies to children such that the number of content children is maximized.

Java Code:

```

java
CopyEdit
import java.util.*;

public class AssignCookies {

```

```

public static int findContentChildren(int[] g, int[] s) {
    Arrays.sort(g);
    Arrays.sort(s);
    int i = 0, j = 0;
    while (i < g.length && j < s.length) {
        if (s[j] >= g[i]) i++;
        j++;
    }
    return i;
}

public static void main(String[] args) {
    int[] g = {1, 2, 3};
    int[] s = {1, 1};
    System.out.println(findContentChildren(g, s)); // Output: 1
}
}

```

Explanation:

1. Sort both arrays.
 2. Assign the smallest sufficient cookie to each child.
 3. Maximize satisfied children.
-

PROBLEM:35

Problem Statement:

Given weights and values of n items, and a capacity W , find the maximum value in a **Fractional Knapsack** where you can take fractions of items.

Java Code:

```

java
CopyEdit
import java.util.*;

class Item {
    int weight, value;
    Item(int v, int w) {
        value = v;
        weight = w;
    }
}

public class FractionalKnapsack {
    public static double fractionalKnapsack(int W, Item[] arr) {
        Arrays.sort(arr, (a, b) -> Double.compare((double)b.value/b.weight,
(double)a.value/a.weight));
        double total = 0.0;
        for (Item item : arr) {
            if (W == 0) break;

```



```

        if (item.weight <= W) {
            total += item.value;
            W -= item.weight;
        } else {
            total += (double)item.value * W / item.weight;
            W = 0;
        }
    }
    return total;
}

public static void main(String[] args) {
    Item[] items = {new Item(60, 10), new Item(100, 20), new Item(120,
30)};
    int W = 50;
    System.out.println(fractionalKnapsack(W, items)); // Output: 240.0
}
}

```

Explanation:

1. Sort items by value/weight ratio.
 2. Pick full items greedily, then fraction of the next.
 3. Return total value.
-

PROBLEM:36

Problem Statement:

Given coin denominations and a value v , find the minimum number of coins to make the change using a greedy approach.

Java Code:

```

java
CopyEdit
import java.util.*;

public class MinCoins {
    public static void findMinCoins(int[] coins, int V) {
        Arrays.sort(coins);
        List<Integer> result = new ArrayList<>();
        for (int i = coins.length - 1; i >= 0 && V > 0; i--) {
            while (V >= coins[i]) {
                V -= coins[i];
                result.add(coins[i]);
            }
        }
        System.out.println("Coins used: " + result);
    }

    public static void main(String[] args) {

```

```
int[] coins = {1, 2, 5, 10, 20, 50, 100, 500, 2000};
int value = 93;
findMinCoins(coins, value); // Output: Coins used: [50, 20, 20, 2,
1]
    }
}
```

Explanation:

1. Sort coins in ascending order.
 2. Start from the largest and subtract greedily.
 3. Continue till the value becomes zero.
-

PROBLEM:37

Problem Statement:

You are given bills in a queue. Each lemonade costs \$5. Customers pay with \$5, \$10, or \$20. Return true if you can provide change for every customer.

Java Code:

```
java
CopyEdit
public class LemonadeChange {
    public static boolean lemonadeChange(int[] bills) {
        int five = 0, ten = 0;
        for (int bill : bills) {
            if (bill == 5) five++;
            else if (bill == 10) {
                if (five == 0) return false;
                five--; ten++;
            } else {
                if (ten > 0 && five > 0) {
                    ten--; five--;
                } else if (five >= 3) {
                    five -= 3;
                } else return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int[] bills = {5, 5, 5, 10, 20};
        System.out.println(lemonadeChange(bills)); // Output: true
    }
}
```

Explanation:

1. Track count of \$5 and \$10 bills.
2. Always try to give change using larger denominations first.
3. Return false if not enough change.

PROBLEM:38

Problem Statement:

Given a string with parentheses, check if they are valid. A string is valid if every opening has a matching closing bracket in the correct order.

Java Code:

```
java
CopyEdit
import java.util.*;

public class ValidParenthesis {
    public static boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();
        for (char ch : s.toCharArray()) {
            if (ch == '(' || ch == '[' || ch == '{') stack.push(ch);
            else {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if ((ch == ')' && top != '(') ||
                    (ch == ']' && top != '[') ||
                    (ch == '}' && top != '{')) return false;
            }
        }
        return stack.isEmpty();
    }

    public static void main(String[] args) {
        String input = "{[()]}";
        System.out.println(isValid(input)); // Output: true
    }
}
```

Explanation:

1. Use a stack to track open brackets.
2. On closing bracket, check top of stack.
3. Return true if balanced at the end.

PROBLEM: 34

Problem Statement:

Write a Java program to find the **Next Smaller Element** for each element in the array.

Java Code:

```
import java.util.*;

public class NextSmallerElement {
    public static int[] nextSmaller(int[] nums) {
        int[] res = new int[nums.length];
        Stack<Integer> stack = new Stack<>();

        for (int i = nums.length - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() >= nums[i])
                stack.pop();
            res[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(nums[i]);
        }

        return res;
    }

    public static void main(String[] args) {
        int[] nums = {4, 8, 5, 2, 25};
        System.out.println("Next Smaller Element: " +
            Arrays.toString(nextSmaller(nums)));
    }
}
```

Explanation:

1. Traverse from right to left.
 2. Use stack to keep smaller values.
 3. Pop greater values and assign the next smaller one.
-

PROBLEM: 35**Problem Statement:**

Find **Number of Next Greater Elements** to the right for each element.

Java Code:

```
import java.util.*;

public class CountNGEsRight {
    public static int[] countNGEs(int[] nums) {
        int[] res = new int[nums.length];
        Stack<Integer> stack = new Stack<>();
    }
```

```

        for (int i = nums.length - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() <= nums[i])
                stack.pop();
            res[i] = stack.size();
            stack.push(nums[i]);
        }

        return res;
    }

    public static void main(String[] args) {
        int[] nums = {4, 5, 2, 10};
        System.out.println("NGE Count: " +
            Arrays.toString(countNGEs(nums)));
    }
}

```

Explanation:

1. Stack tracks greater elements.
 2. At each position, count how many greater values are right to it.
-

PROBLEM: 36

Problem Statement:

Trapping Rainwater problem: Find the amount of water that can be trapped.

Java Code:

```

public class TrappingRainWater {
    public static int trap(int[] height) {
        int n = height.length, left = 0, right = n - 1, leftMax = 0,
        rightMax = 0, water = 0;
        while (left < right) {
            if (height[left] <= height[right]) {
                leftMax = Math.max(leftMax, height[left]);
                water += leftMax - height[left];
                left++;
            } else {
                rightMax = Math.max(rightMax, height[right]);
                water += rightMax - height[right];
                right--;
            }
        }
        return water;
    }

    public static void main(String[] args) {
        int[] height = {4, 2, 0, 3, 2, 5};
        System.out.println("Trapped Water: " + trap(height));
    }
}

```

```
}
```

Explanation:

1. Two pointers from both ends.
 2. Maintain max height from left and right.
 3. Water at each index = $\min(\text{leftMax}, \text{rightMax}) - \text{height}[i]$.
-

PROBLEM:38

Problem Statement:

Write a Java program to calculate the **sum of minimums of all subarrays** of a given integer array.

Java Code:

```
java
CopyEdit
import java.util.*;

public class SubarrayMinimumSum {
    public static void main(String[] args) {
        int[] arr = {3, 1, 2, 4};
        System.out.println("Sum of subarray minimums: " +
sumSubarrayMins(arr));
    }

    public static int sumSubarrayMins(int[] arr) {
        int mod = 1_000_000_007;
        int n = arr.length;
        Stack<Integer> stack = new Stack<>();
        int[] prev = new int[n];
        int[] next = new int[n];

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && arr[stack.peek()] > arr[i])
stack.pop();
            prev[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(i);
        }

        stack.clear();

        for (int i = n - 1; i >= 0; i--) {
            while (!stack.isEmpty() && arr[stack.peek()] >= arr[i])
stack.pop();
            next[i] = stack.isEmpty() ? n : stack.peek();
            stack.push(i);
        }

        long result = 0;
        for (int i = 0; i < n; i++) {
```

```

        long count = (i - prev[i]) * (next[i] - i);
        result = (result + arr[i] * count) % mod;
    }
    return (int) result;
}
}

```

Explanation:

1. For each element, we find how many subarrays it's the minimum for using monotonic stacks.
2. `prev[i]` gives the previous lesser element's index.
3. `next[i]` gives the next lesser or equal element's index.
4. Total subarrays where `arr[i]` is minimum: $(i - \text{prev}[i]) * (\text{next}[i] - i)$.
5. We multiply this count with `arr[i]` and add to result.

6. PROBLEM:39

7. Problem Statement:

Given n meetings with start and end times, schedule the maximum number of non-overlapping meetings in one room.

8.

9. Java Code:

```

10. java
11. CopyEdit
12. import java.util.*;
13.
14. class Meeting {
15.     int start, end;
16.     Meeting(int s, int e) {
17.         start = s;
18.         end = e;
19.     }
20. }
21.
22. public class NMeetings {
23.     public static int maxMeetings(int[] start, int[] end, int n) {
24.         List<Meeting> meetings = new ArrayList<>();
25.         for (int i = 0; i < n; i++) meetings.add(new
Meeting(start[i], end[i]));
26.         meetings.sort((a, b) -> a.end - b.end);
27.         int count = 1;
28.         int lastEnd = meetings.get(0).end;
29.         for (int i = 1; i < n; i++) {
30.             if (meetings.get(i).start > lastEnd) {
31.                 count++;
32.                 lastEnd = meetings.get(i).end;
33.             }
34.         }
35.         return count;
36.     }
37.
38.     public static void main(String[] args) {
39.         int[] start = {1, 3, 0, 5, 8, 5};
40.         int[] end = {2, 4, 6, 7, 9, 9};
41.         System.out.println(maxMeetings(start, end, start.length));
42.         // Output: 4
43.     }

```

44. _____

45. Explanation:

Sort by end time, always choose the next meeting with a start time after the previous end.

46. _____

47. PROBLEM:40

48. Problem Statement:

You are given an array where each element represents your max jump. Return if you can reach the end.

49. _____

50. Java Code:

```
51. java
52. CopyEdit
53. public class JumpGame {
54.     public static boolean canJump(int[] nums) {
55.         int maxReach = 0;
56.         for (int i = 0; i < nums.length; i++) {
57.             if (i > maxReach) return false;
58.             maxReach = Math.max(maxReach, i + nums[i]);
59.         }
60.         return true;
61.     }
62.
63.     public static void main(String[] args) {
64.         int[] nums = {2,3,1,1,4};
65.         System.out.println(canJump(nums)); // Output: true
66.     }
67. }
```

68. _____

69. Explanation:

Track the farthest you can jump at each step. If you can't move further, return false.

70. _____

71. PROBLEM:41

72. Problem Statement:

Given the same array, return the minimum number of jumps to reach the end.

73. _____

74. Java Code:

```
75. java
76. CopyEdit
77. public class JumpGame2 {
78.     public static int jump(int[] nums) {
79.         int jumps = 0, currEnd = 0, currFarthest = 0;
80.         for (int i = 0; i < nums.length - 1; i++) {
81.             currFarthest = Math.max(currFarthest, i + nums[i]);
82.             if (i == currEnd) {
83.                 jumps++;
84.                 currEnd = currFarthest;
85.             }
86.         }
87.         return jumps;
88.     }
89.
90.     public static void main(String[] args) {
91.         int[] nums = {2,3,1,1,4};
92.         System.out.println(jump(nums)); // Output: 2
93.     }
```


94. }

95.

96. Explanation:

Use BFS-style greedy approach. Jump whenever you reach the current farthest jump range.

97.

98. PROBLEM:42

99. Problem Statement:

Given arrival and departure times of trains, return the minimum number of platforms needed.

100.

101. Java Code:

```
102. java
103. CopyEdit
104. import java.util.*;
105.
106. public class MinPlatforms {
107.     public static int findPlatform(int[] arr, int[] dep, int n) {
108.         Arrays.sort(arr);
109.         Arrays.sort(dep);
110.         int i = 1, j = 0, platforms = 1, max = 1;
111.         while (i < n && j < n) {
112.             if (arr[i] <= dep[j]) {
113.                 platforms++;
114.                 i++;
115.             } else {
116.                 platforms--;
117.                 j++;
118.             }
119.             max = Math.max(max, platforms);
120.         }
121.         return max;
122.     }
123.
124.     public static void main(String[] args) {
125.         int[] arr = {900, 940, 950, 1100, 1500, 1800};
126.         int[] dep = {910, 1200, 1120, 1130, 1900, 2000};
127.         System.out.println(findPlatform(arr, dep, arr.length)); //
Output: 3
128.     }
129. }
```

130.

131. Explanation:

Sort arrivals and departures. Track overlapping intervals to find max platforms needed.

132.

133. PROBLEM:43

134. Problem Statement:

Given jobs with deadlines and profits, schedule jobs to maximize total profit using a greedy approach.

135.

136. Java Code: import java.util.*;

```
137. java
138. CopyEdit
139. class Job {
140.     int id, deadline, profit;
141.     Job(int i, int d, int p) {
```

```

142.         id = i; deadline = d; profit = p;
143.     }
144. }
145.
146. public class JobSequencing {
147.     public static int[] jobScheduling(Job[] jobs, int n) {
148.         Arrays.sort(jobs, (a, b) -> b.profit - a.profit);
149.         int maxDeadline = 0;
150.         for (Job job : jobs) maxDeadline = Math.max(maxDeadline,
job.deadline);
151.         int[] result = new int[2]; // [count, profit]
152.         boolean[] slot = new boolean[maxDeadline + 1];
153.
154.         for (Job job : jobs) {
155.             for (int j = job.deadline; j > 0; j--) {
156.                 if (!slot[j]) {
157.                     slot[j] = true;
158.                     result[0]++;
159.                     result[1] += job.profit;
160.                     break;
161.                 }
162.             }
163.         }
164.         return result;
165.     }
166.
167.     public static void main(String[] args) {
168.         Job[] jobs = {
169.             new Job(1, 2, 100),
170.             new Job(2, 1, 19),
171.             new Job(3, 2, 27),
172.             new Job(4, 1, 25),
173.             new Job(5, 3, 15)
174.         };
175.         int[] res = jobScheduling(jobs, jobs.length);
176.         System.out.println("Jobs done: " + res[0] + ", Total profit:
" + res[1]); // Output: 2 127
177.     }
178. }

```

179.

180. **Explanation:**

Sort by profit. For each job, place it in the latest available slot before its deadline.

181.

182. **PROBLEM:44**

183. **Problem Statement:**

Distribute candies to children such that each gets at least one candy and children with higher ratings get more than their neighbors.

184.

185. **Java Code:**

```

186. java
187. CopyEdit
188. public class CandyDistribution {
189.     public static int candy(int[] ratings) {
190.         int n = ratings.length;
191.         int[] candies = new int[n];
192.         Arrays.fill(candies, 1);
193.
194.         for (int i = 1; i < n; i++)
195.             if (ratings[i] > ratings[i-1])

```

```

196.             candies[i] = candies[i-1] + 1;
197.
198.         for (int i = n - 2; i >= 0; i--)
199.             if (ratings[i] > ratings[i+1])
200.                 candies[i] = Math.max(candies[i], candies[i+1] + 1);
201.
202.         int total = 0;
203.         for (int c : candies) total += c;
204.         return total;
205.     }
206.
207.     public static void main(String[] args) {
208.         int[] ratings = {1, 0, 2};
209.         System.out.println(candy(ratings)); // Output: 5
210.     }
211. }
212.
213.     Explanation:

```

Use two passes: left to right and right to left, ensuring rating rules are maintained.

PROBLEM:45

Problem Statement:

Program for **Shortest Job First (SJF)** CPU Scheduling — Given processes with burst times, schedule them to minimize average waiting time using non-preemptive SJF.

Java Code:

```

java
CopyEdit
import java.util.*;

class Process {
    int id, burst;
    Process(int id, int burst) {
        this.id = id;
        this.burst = burst;
    }
}

public class SJF {
    public static void main(String[] args) {
        Process[] processes = {
            new Process(1, 6),
            new Process(2, 8),
            new Process(3, 7),
            new Process(4, 3)
        };

        Arrays.sort(processes, Comparator.comparingInt(p -> p.burst));

        int totalWait = 0, wait = 0;
        for (int i = 0; i < processes.length; i++) {

```

```

        System.out.println("Process " + processes[i].id + " waited " +
wait + " units.");
        totalWait += wait;
        wait += processes[i].burst;
    }

    double avg = (double) totalWait / processes.length;
    System.out.println("Average Waiting Time: " + avg);    // Output:
5.25
    }
}

```

Explanation:

Sort processes by burst time. Accumulate waiting time as you schedule each one.

PROBLEM:46

Problem Statement:

Implement **Least Recently Used (LRU) Page Replacement Algorithm** using a doubly linked list and HashMap.

Java Code:

```

java
CopyEdit
import java.util.*;

class LRUCache {
    int cap;
    LinkedHashMap<Integer, Integer> map;

    public LRUCache(int capacity) {
        this.cap = capacity;
        map = new LinkedHashMap<>(capacity, 0.75f, true);
    }

    public int get(int key) {
        return map.getOrDefault(key, -1);
    }

    public void put(int key, int value) {
        if (map.containsKey(key)) {
            map.put(key, value);
        } else {
            if (map.size() == cap) {
                int lru = map.keySet().iterator().next();
                map.remove(lru);
            }
            map.put(key, value);
        }
    }
}

```

```

public static void main(String[] args) {
    LRUCache lru = new LRUCache(2);
    lru.put(1, 1);
    lru.put(2, 2);
    System.out.println(lru.get(1)); // 1
    lru.put(3, 3); // evicts key 2
    System.out.println(lru.get(2)); // -1
}
}

```

Explanation:

Use LinkedHashMap with access-order enabled. When full, remove least recently used key.

PROBLEM:47

Problem Statement:

Insert a new interval into a list of non-overlapping intervals and merge if necessary.

Java Code:

```

java
CopyEdit
import java.util.*;

public class InsertInterval {
    public static int[][] insert(int[][] intervals, int[] newInterval) {
        List<int[]> res = new ArrayList<>();
        int i = 0;

        // Before overlap
        while (i < intervals.length && intervals[i][1] < newInterval[0])
            res.add(intervals[i++]);

        // Merge
        while (i < intervals.length && intervals[i][0] <= newInterval[1]) {
            newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
            newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
            i++;
        }
        res.add(newInterval);

        // After merge
        while (i < intervals.length)
            res.add(intervals[i++]);

        return res.toArray(new int[res.size()][2]);
    }

    public static void main(String[] args) {
        int[][] intervals = {{1,3}, {6,9}};
        int[] newInterval = {2,5};
        int[][] result = insert(intervals, newInterval);
    }
}

```

```
        for (int[] i : result) System.out.println(Arrays.toString(i)); //
[[1,5], [6,9]]
    }
}
```

Explanation:

Add intervals before the new one, merge overlapping, then add remaining.

PROBLEM:48**Problem Statement:**

Merge overlapping intervals from a given list.

Java Code:

```
java
CopyEdit
import java.util.*;

public class MergeIntervals {
    public static int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
        List<int[]> res = new ArrayList<>();

        int[] curr = intervals[0];
        for (int[] interval : intervals) {
            if (interval[0] <= curr[1]) {
                curr[1] = Math.max(curr[1], interval[1]);
            } else {
                res.add(curr);
                curr = interval;
            }
        }
        res.add(curr);
        return res.toArray(new int[res.size()][]);
    }

    public static void main(String[] args) {
        int[][] intervals = {{1,3}, {2,6}, {8,10}, {15,18}};
        int[][] result = merge(intervals);
        for (int[] i : result) System.out.println(Arrays.toString(i)); //
[[1,6], [8,10], [15,18]]
    }
}
```

Explanation:

Sort by start time and merge overlapping intervals by extending the end.

PROBLEM:49

Problem Statement:

Return the minimum number of intervals you need to remove to make the rest non-overlapping.

Java Code:

```
java
CopyEdit
import java.util.*;

public class NonOverlappingIntervals {
    public static int eraseOverlapIntervals(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[1] - b[1]);
        int count = 0;
        int end = Integer.MIN_VALUE;

        for (int[] i : intervals) {
            if (i[0] >= end) {
                end = i[1];
            } else {
                count++;
            }
        }
        return count;
    }

    public static void main(String[] args) {
        int[][] intervals = {{1,2}, {2,3}, {3,4}, {1,3}};
        System.out.println(eraseOverlapIntervals(intervals)); // Output: 1
    }
}
```

Explanation:

Sort by end time, and greedily pick non-overlapping intervals.

Congratulations on completing the PDF created by SYNTAX ERROR, also known as Abhishek Rathor! Your dedication and perseverance are truly commendable. For more insights and updates, feel free to connect on Instagram at SYNTAX ERROR. Keep up the great work!